



DIGITAL LOGIC DESIGN

Design and Simulation of 8-Bit Signed Binary Number Representation

Using the Sign-Magnitude Method

BY TEAM MEMBERS:

2620030230_Ramya Sree

2620030178_Manvitha

2620030188_Bhargav Karthik

2620030194_Shiva Sai

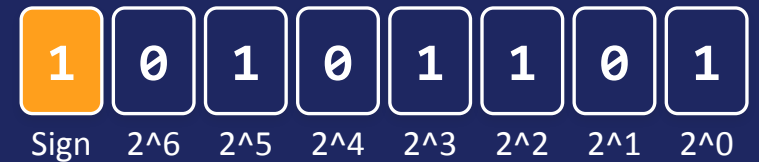
A study of representing and simulating positive & negative integers
within an 8-bit binary word

Introduction & Objective

Digital systems store everything — including negative numbers — as bits. Since plain binary only expresses magnitude, a dedicated scheme is needed to also encode the sign of a value. The Sign-Magnitude method is the most intuitive of these schemes, closely mirroring how humans write signed numbers on paper (a '+' or '-' followed by the value).

- 1 Represent**
Encode both positive and negative integers within a fixed 8-bit word.
- 2 Design**
Define the sign-magnitude bit structure and its encoding/decoding rules.
- 3 Simulate**
Model the scheme in software/logic to verify range, zero cases and arithmetic.

AN 8-BIT WORD



MSB = Sign Bit

Remaining 7 Bits

0 = Positive 1 = Negative Represent the Magnitude

“One bit for sign, the rest for value — closes the gap between binary and everyday signed notation.”

Ways to Represent Signed Numbers

Three common binary schemes — this project focuses on the first

FOCUS OF THIS DECK

Sign-Magnitude

MSB stores the sign (0/1); remaining bits store the magnitude, unchanged from unsigned binary.

-5 → **1 0000101**

1's Complement

Negative numbers formed by inverting every bit of the positive value.

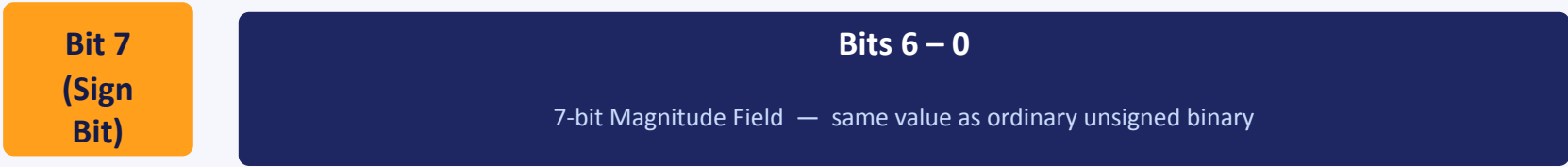
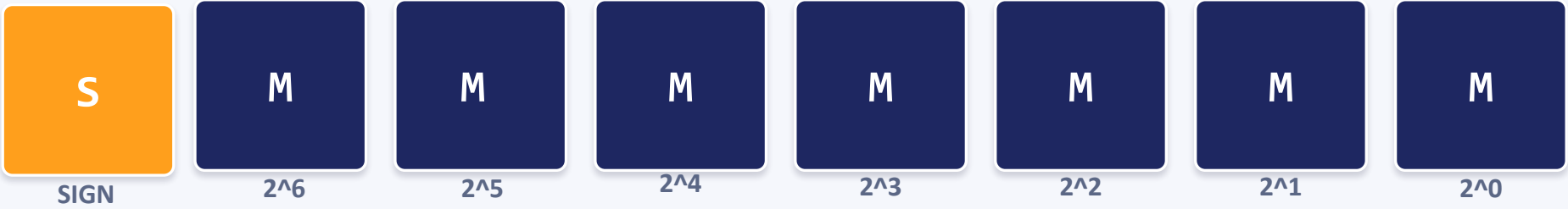
-5 → **1 1111010**

2's Complement

Negative numbers formed by inverting all bits, then adding 1. Used in most real hardware.

-5 → **1 1111011**

Structure of the Sign-Magnitude Format

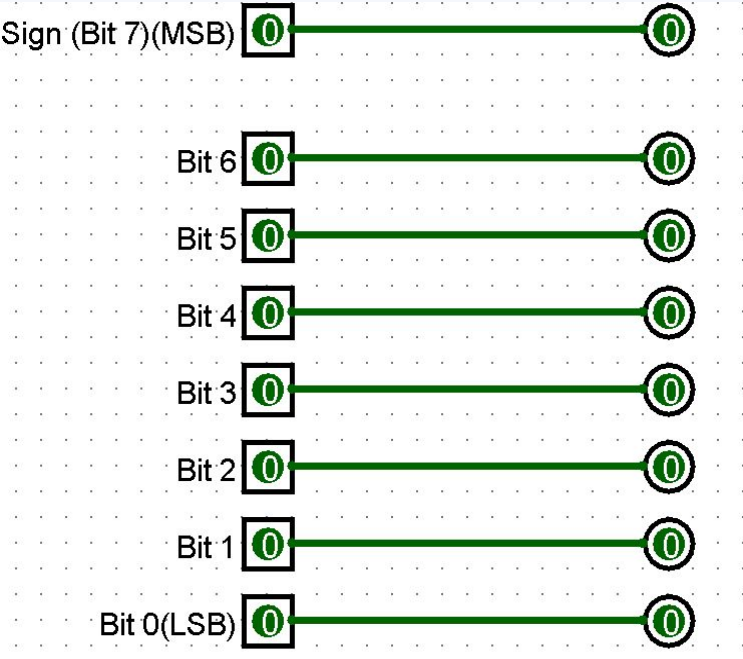


0 Sign Bit = 0

The value is positive.
Example: 0 0000101 = +5

1 Sign Bit = 1

The value is negative.
Example: 1 0000101 = -5



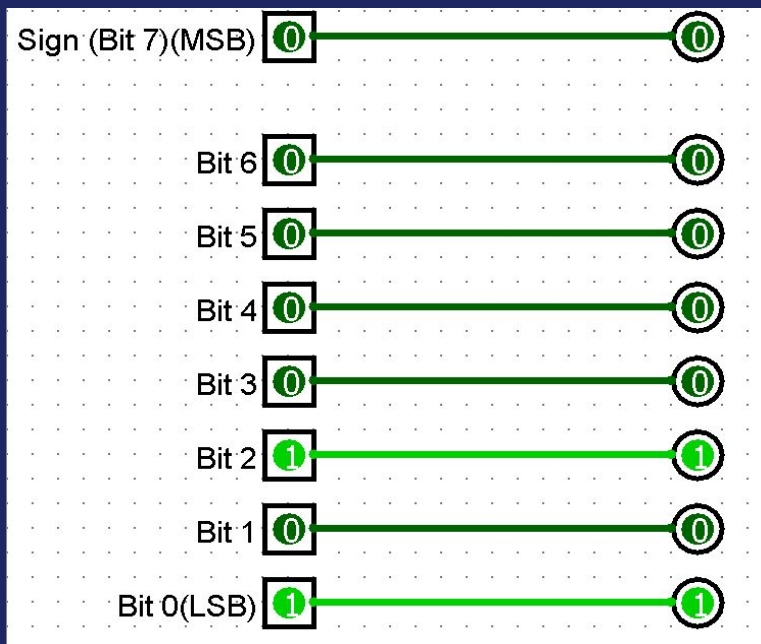
General form: $N = (-1)^{SIGN} \times |Magnitude|$, where the sign bit selects the exponent of -1

Encoding Examples

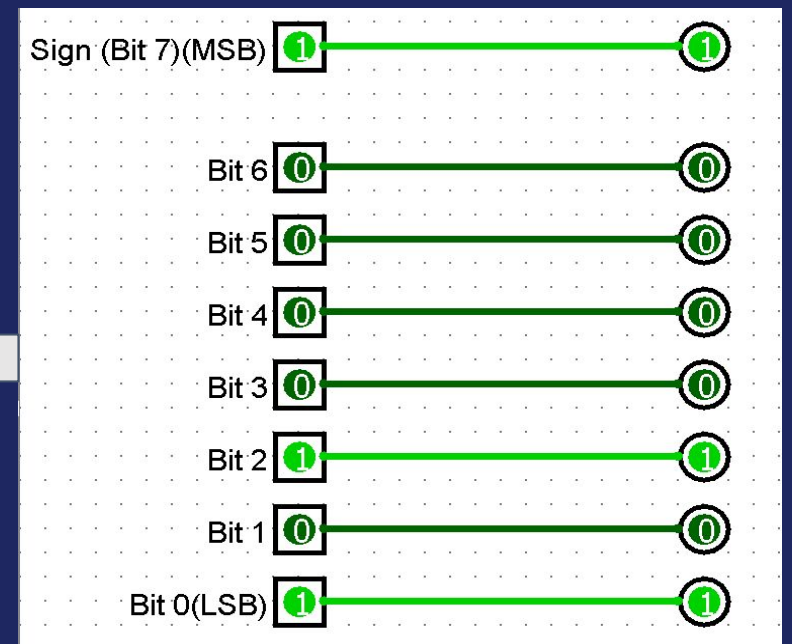
Decimal values converted into 8-bit sign-magnitude form

Decimal	Sign	Magnitude (7 bits)	8-bit Sign-Magnitude	Note
+5	0	0000101	0 0000101	positive
-5	1	0000101	1 0000101	negative
+127	0	1111111	0 1111111	max positive
-127	1	1111111	1 1111111	max negative
+0	0	0000000	0 0000000	zero (positive)
-0	1	0000000	1 0000000	zero (negative)

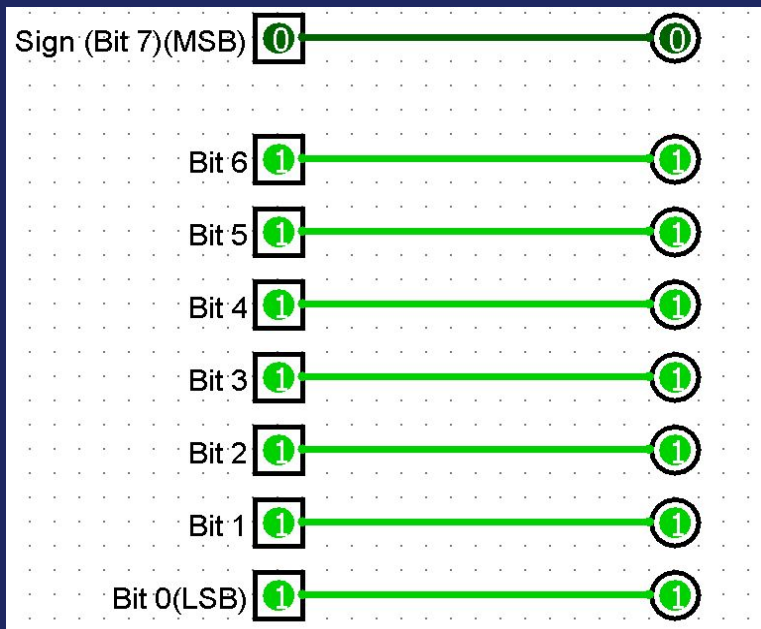
Notice: +0 and -0 have two different bit patterns for the same value — a key quirk of this method (see next slide).



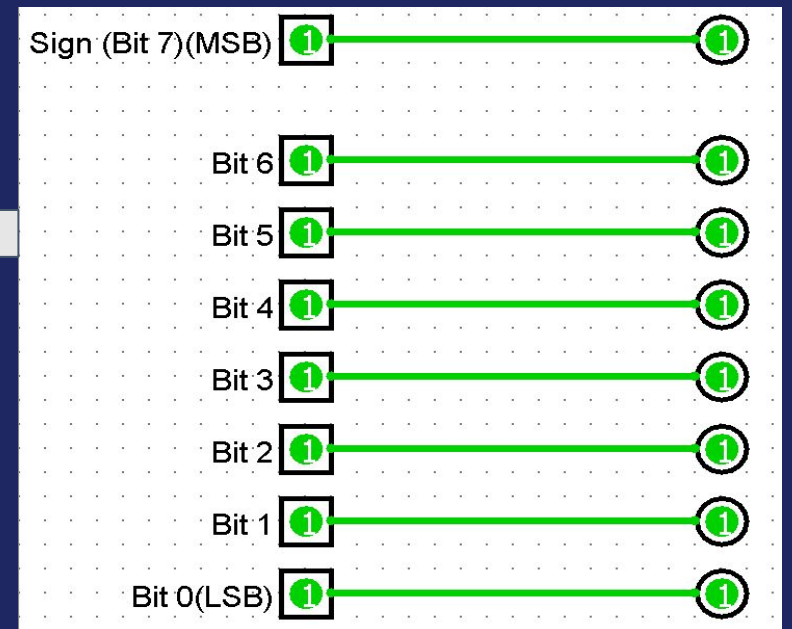
→ +5



← -5



→ +127



← -127

Range and Special Cases

What 8 bits can and cannot cleanly express

7

magnitude bits
available for value

-127 to +127

representable
decimal range

256

total bit patterns,
only 255 distinct values

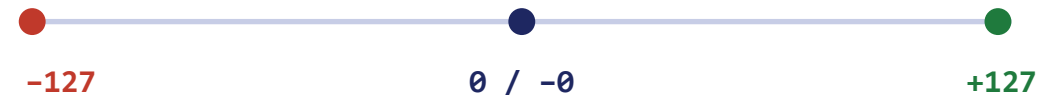
The Duplicate-Zero Problem

0 0 0 0 0 0 0 0 = +0

1 0 0 0 0 0 0 0 = -0

Two encodings map to the same value, wasting a bit pattern and complicating equality checks in simulation.

Symmetric Range



Unlike 2's complement (-128 to +127), sign-magnitude is perfectly symmetric — at the cost of the duplicate zero above.

Simulating Sign-Magnitude Arithmetic

Unlike 2's complement, sign-magnitude cannot simply add bit patterns — the simulation logic must branch on the operand signs before performing magnitude arithmetic.

1

Read Signs

Extract the MSB of both operands to identify each number's sign.

2

Compare Signs

If signs match, add the magnitudes and keep the common sign.

3

Subtract if Differ

If signs differ, subtract the smaller magnitude from the larger.

4

Assign Result Sign

Result takes the sign of the operand with the larger magnitude.

5

Check Overflow

Flag overflow if the magnitude result exceeds 7-bit capacity (127).

WORKED EXAMPLE

$(+5) + (-3)$: signs differ → subtract magnitudes $5 - 3 = 2$ → sign of larger magnitude (+5) is positive → result = +2

0 0000101 + 1 0000011 → 0 0000010

Advantages, Limitations & Conclusion

Advantages

- ✓ Simple, intuitive mapping to sign + value notation
- ✓ Symmetric range: equal count of positive & negative values
- ✓ Easy manual conversion between decimal and binary
- ✓ Sign bit is trivially isolated by reading the MSB

Limitations

- ✗ Two representations of zero (+0 and -0)
- ✗ Addition/subtraction needs conditional sign logic, not direct bit addition
- ✗ Hardware adders are more complex than 2's complement designs
- ✗ Rarely used in real processors — mainly educational value

Conclusion

The sign-magnitude method offers a clear, human-readable foundation for understanding signed binary representation. Simulation confirms its correct range (-127 to +127) and highlights the practical issues — duplicate zero and added arithmetic complexity — that led modern hardware to adopt 2's complement instead.

*Thank
you!*

